

Cover Page

Course Title: **STUDY PROJECT (BCS ZC241T)**

Project Title: *Smart Workflow-Based Queue and Administrative Service Management System for Educational Institutions*

Student Name(s):

1. BHUPESH KUMAR
2. GONE MANJUSRI
3. HARSH UPADHYAY
4. HARANGAD SINGH GHAI

Student ID(s):

1. 2023ebcs402
2. 2023ebcs404
3. 2023ebcs445
4. 2023ebcs526

Project Advisor / Supervisor: **DHRUVIKA CHAUHAN**

Date of Submission:

1. Project Idea Summary

Abstract

Educational institutions provide various administrative services such as admission processing, exam registration, certificate issuance, ID card services, and student consultation. In many colleges, these services are still handled using physical counters or partially digital systems. This often results in long waiting times, overcrowding, repeated visits by students, and missed deadlines. Students and parents also face confusion due to unclear service requirements and lack of proper guidance.

The aim of this project is to design a smart administrative service management system that improves the existing process by introducing structured workflows and online queue management. The proposed system allows students and parents to understand service requirements in advance, ask questions using an AI chatbot assistant, and join a virtual queue or book an appointment as required. Each service request is processed step by step through a defined workflow, ensuring transparency and reducing errors.

The system also provides dashboards for administrative staff to manage requests efficiently and track progress. The project is developed using the MERN technology stack, which is widely used and easy to maintain. This solution helps reduce waiting time, minimize repeated visits, and improve the overall administrative experience in educational institutions.

2. Project Background & Motivation

2.1 Background

Colleges and schools provide many administrative services such as admissions, exam registration, ID/admit cards, certificate issuance, and consultation appointments. Traditionally these services have been handled at physical counters with paper registers and face-to-face processing. Over time campuses added partial digital tools (online forms, payment portals), but the actual verification and approval steps often remained manual. This mix of old and new creates confusion, slows processing, and causes repeated visits by students and parents. The project document provides several examples and flow diagrams showing where delays and errors happen in current practice.

2.2 Problem statement

- **Core problem:** Administrative services at many educational institutions are multi-step and deadline-sensitive, but they are treated as single-step tasks in practice. This mismatch causes inefficiency and poor user experience.
- **Who is affected:** Students, parents, administrative staff (clerks, officers), and institute management.

- **Where it occurs:** Admission counters, exam cells, accounts, certificate desks, ID/admit card offices, and parts of the online portal that still require physical verification.
- **Typical symptoms:**
 - Long waiting lines during peak periods (admission and exam windows).
 - Students returning multiple times because required documents or fees were not checked in advance.
 - Lack of a single place to track the progress of a request or to see which step is pending.
 - Staff spend time answering the same routine questions instead of processing requests.
 - Critical deadlines (exam registration, admission confirmation) are sometimes missed due to poor prioritization.

2.3 Causes of the problem

- Processes are fragmented across departments and rely on human coordination.
- Partial digitization only captures data entry, not the end-to-end workflow.
- Commercial queue tools focus on crowd control, not multi-step academic workflows.
- Students lack clear pre-visit guidance and sample documents, which leads to failed attempts at counters.

2.4 Motivation — why we chose this problem

- **High impact on students:** Administrative delays directly affect exam eligibility, admissions, scholarships, and graduation timelines. Reducing those delays helps students academically and reduces stress for families.
- **Visible and measurable:** The problem is easy to observe (queues, returns) and can be measured with simple KPIs (wait time, repeat visits), so improvements are demonstrable.
- **Feasible for a semester project:** The solution can be prototyped using web technologies and rule-based assistants and tested in a small pilot without replacing campus ERPs.
- **Research and practice gap:** Literature and university pilots show that virtual queuing reduces crowding but does not fix workflow failures. There is a gap for a solution that combines hybrid queuing, workflow modeling, and pre-service guidance — exactly what this project targets.

2.5 What we aim to achieve

- Reduce average visible wait time and the number of repeat visits.
- Provide clear pre-visit guidance so users prepare correct documents.
- Model each administrative service as a stepwise workflow so partial progress is visible.
- Prioritize requests that are close to academic deadlines.
- Reduce routine queries to staff using a chatbot assistant.

3. Educational Value and Course Alignment

3.1 Relevance to Course Objectives

This project is directly aligned with the objectives of the course, as it applies theoretical concepts taught in class to a real-world problem faced by educational institutions. The project design and implementation reflect the topics, tools, and practices discussed during the course.

Alignment with Course Topics

- **Web Application Development:**
The project uses the MERN stack (MongoDB, Express.js, React, Node.js), which is commonly taught in modern web development courses.
- **Backend Systems & APIs:**
REST APIs are designed using Node.js and Express to handle services, queues, workflows, and chatbot requests.
- **Database Systems:**
MongoDB is used to store service requests, workflow steps, user data, and logs, aligning with database design and querying concepts.
- **Caching and Queue Management:**
Redis and BullMQ are used to manage queues, background jobs, and performance optimization, which are important backend concepts.
- **System Design Concepts:**
The project applies workflow modeling, modular architecture, and separation of concerns as discussed in system design topics.
- **AI and Emerging Technologies:**
A chatbot is implemented using the **Gemini API** to assist users with service-related queries, introducing students to practical use of AI APIs.

Skills and Concepts Taught

- Designing scalable full-stack applications.
- Implementing queue-based and workflow-driven systems.
- Integrating third-party APIs such as the Gemini AI API.
- Handling asynchronous tasks using background job queues.
- Applying software engineering best practices such as modular code and clean architecture.

Practical Application of Theory

- Workflow diagrams created in the project document are converted into real backend logic.
- Queueing theory concepts are applied using Redis and BullMQ.
- Database concepts are used to track request states and audit logs.
- AI theory is applied practically by using Gemini API for chatbot-based assistance.
- Frontend concepts are used to design user-friendly portals and dashboards.

3.2 Learning Outcomes

After completing this project, students are expected to achieve the following learning outcomes:

Technical Skills

- Build full-stack applications using the MERN technology stack.
- Develop RESTful APIs using Node.js and Express.
- Design and manage MongoDB schemas for workflow-based systems.
- Use Redis for caching and BullMQ for queue and background job processing.
- Integrate AI services using the **Gemini API** to build a chatbot assistant.
- Implement basic security measures such as role-based access control.

Analytical and Design Skills

- Analyze real administrative problems and convert them into technical solutions.
- Design workflow-based systems using flow diagrams and step-wise processing.
- Define and measure KPIs such as waiting time and repeat visits.
- Evaluate system performance and identify process bottlenecks.

Tool and Framework Exposure

- **Frontend:** React, Zustand, Tailwind CSS, ShadCN UI, Ant Design (if required).
- **Backend:** Node.js, Express.js.
- **Database:** MongoDB.
- **Caching & Queue:** Redis, BullMQ.
- **AI Tool:** Gemini API for chatbot-based user interaction.
- **Development Tools:** Git, Postman, basic deployment tools.

Problem-Solving and Documentation Skills

- Break down complex administrative workflows into manageable components.
- Document system architecture, workflows, and APIs clearly.
- Prepare formal project reports and presentations.
- Communicate technical ideas effectively during reviews and viva.

4 Objectives

4.1 Primary Objectives

- Reduce waiting time for students and parents during administrative processes.
- Minimize repeated visits by providing clear service guidance and workflow-based processing.
- Implement a hybrid queue system (virtual queue and appointment booking).
- Model administrative services as step-by-step workflows.
- Provide an AI chatbot (Gemini API) to assist users with service-related queries.
- Enable administrators to monitor requests using dashboards and audit logs.

4.2 Secondary Objectives

- Improve clarity of administrative processes by clearly defining service steps and requirements.
- Reduce manual workload on administrative staff by automating queue handling and request tracking.
- Provide transparency to users by showing real-time request status.
- Improve communication between students and administration using notifications and chatbot support.
- Collect basic operational data (wait time, request volume) to understand administrative bottlenecks.
- Demonstrate a practical working prototype suitable for academic evaluation.

5. Research and Analysis

5.1 Existing Solutions

Types currently used on campuses

- Manual counters and token systems (paper).
- Semi-digital: online forms and payment portals where verification stays manual
- Campus ERPs (central records but rigid workflows and poor student UX).
- Commercial queue/appointment vendors (QueueHub, Qwaiting, Qless, Waitwhile, QueueBee) that provide tokening, appointments and notifications.

Main limitations / gaps

- Vendor systems mostly assume **single-step services** (bank/retail model) and treat a request as one transaction. This fails for multi-step academic services.
- **Deadline-aware prioritization** (exam cutoffs) is rare in off-the-shelf tools.
- **Pre-service checklist validation** and document-level guidance are not well supported; chatbots in vendor literature mostly handle simple FAQs.
- **Customization cost and ERP integration friction** limit adoption for small colleges.

5.2 Functional Requirements

User-facing

- Service catalogue pages (documents, eligibility, deadlines).
- AI Chatbot (Gemini API) to answer service questions, validate checklists, and guide users.
- Join virtual queue (token) or book appointment.
- View request status and workflow step progress.
- Receive email notifications (token called, appointment reminder, completion).

Staff/Admin

- Claim and process workflow steps (verify, approve, reject with comments).
- Dashboard showing active workflows, SLA alerts, bottlenecks, repeat-visit counts.
- Per-request audit logs (who did what, when).
- Export completed records as CSV and minimal REST API for ERP sync.

System

- Workflow engine (configurable steps per service).
- Queue scheduler (BullMQ + Redis) and background workers.
- Persistent storage of requests (MongoDB) and cache (Redis).
- Authentication & role-based access control.
- Logging, monitoring and basic metrics export.

5.3 Non-Functional Requirements

Usability

- Simple, mobile-responsive UI (React + Tailwind), clear service pages, easy chatbot access.
- Low friction for joining queue (no heavy login required for basic token).

Performance

- ETA and token calculation within seconds.
- Support pilot load (100–200 requests over pilot) with responsive dashboards.

Security

- HTTPS/TLS for all endpoints.
- Role-based access (students, staff, admin).
- Store PII encrypted at rest where required; log actions for audit.
- Secure API keys for Gemini providers.

Scalability

- Redis + BullMQ design allows horizontal worker scaling.
- MongoDB can be scaled vertically or sharded if needed.
- System designed to scale to larger student populations after pilot.

Maintainability

- Modular codebase (frontend/backend separation), documented API contracts, unit tests for core modules.
- Configuration-driven workflow templates for easy changes without code edits.

Reliability / Availability

- Job retry logic for background tasks; DB backups during pilot.
- Reasonable uptime during business hours for campus operations.

5.4 Feasibility Analysis

Technical Feasibility

- Uses standard technologies: MERN stack, Redis, BullMQ, and Gemini API.
- All tools are well-documented and easy to implement.
- System is modular and suitable for partial implementation.

Time Feasibility

- **Phase 1 (05 Jan – 02 Feb 2026):** Research and documentation tasks are manageable.
- **Phase 2 (03 Feb – 15 Feb 2026):** Design and PoC can be completed.
- **Phase 3 (16 Feb – 03 Mar 2026):** Partial implementation and validation are feasible.

Resource Feasibility

- Developed by a small team(4 member).
- Requires only basic hardware, internet, and limited API usage.

Proposed High-Level Solution

Overview

The proposed solution is a **workflow-based administrative service management system** designed for educational institutions. It aims to reduce waiting time, repeated visits, and confusion by structuring administrative services into clear steps and supporting them with a hybrid queue system and an intelligent chatbot assistant. The system is developed using the MERN stack, Redis with BullMQ for queue handling, and a RAG-based chatbot using the Gemini API.

System Components

Student / Parent Portal

- View administrative services with required documents, eligibility rules, and deadlines.
- Interact with an **AI chatbot (RAG-based using Gemini API)** for service-related questions.

- Join a virtual queue or book an appointment.
- Track the status of their service request.

Staff / Admin Dashboard

- View and process service requests.
- Verify documents and update workflow steps.
- Approve or reject requests with remarks.
- Manage queue flow and handle priority cases.

Backend System

- REST APIs built using Node.js and Express.
- Workflow engine to process each service step by step.
- Queue and background job handling using Redis and BullMQ.
- Secure chatbot integration layer.
- Authentication and role-based access control.

Chatbot

The chatbot follows a **Retrieval-Augmented Generation (RAG)** approach:

- Service rules, FAQs, and document guidelines are stored in a knowledge base.
- Relevant information is retrieved based on user queries.
- Gemini API generates accurate, context-aware responses.
- The chatbot guides users before they join the queue, reducing incorrect or incomplete requests.

Request Flow (High Level)

1. User selects a service.
2. User interacts with the RAG chatbot for guidance.
3. User joins a queue or books an appointment.
4. A workflow instance is created in the system.
5. Staff/Admin processes the request step by step.
6. User is notified upon completion.

Technology Stack

- **Frontend:** React, Zustand, Tailwind CSS, ShadCN UI, Ant Design (if required)
- **Backend:** Node.js, Express.js
- **Database:** MongoDB
- **Queue & Cache:** Redis, BullMQ
- **Chatbot:** RAG-based chatbot using Gemini API

6. Project Scope and Expected Deliverables

6.1 Scope Definition

Included (Phase-1)

- Pilot services: Admission, Exam Registration, Admit/ID Card, Certificate Issuance, Consultation appointments.
- Student/Parent portal with service details and **chatbot (Gemini API)** for guidance.
- Virtual queue and appointment booking.
- Single **Staff/Admin dashboard** to process requests and manage workflows.
- Workflow-based service processing and basic notifications.
- Use of MERN stack, Redis, BullMQ.

Excluded (Phase-1)

- Full ERP integration or replacement.
- Online fee payment gateway.
- Mobile application development.
- Advanced AI/ML model training.
- Full production deployment.

Assumptions & Constraints

- Limited pilot users and sample data.
- Semester-bound timeline and limited resources.
- Basic familiarity with MERN stack technologies.

6.2 Deliverables (Phase-1)

- Project proposal / Phase-1 report.
- Problem statement and objectives.
- Literature review and background study.
- Existing system analysis.
- Proposed high-level system solution.
- Scope, constraints, feasibility analysis, and timeline.

7. Preliminary Project Timeline and Milestones

Overall Phases (high level)

- **Phase 1 — Problem Identification & Planning:** 05 Jan 2026 – 02 Feb 2026
- **Phase 2 — Design & Proof of Concept:** 03 Feb 2026 – 15 Feb 2026
- **Phase 3 — Implementation Readiness & Validation:** 16 Feb 2026 – 03 Mar 2026

Phase-1 (05 Jan – 02 Feb 2026) — Week-by-week activities & milestones

- **Week 0 (05 Jan – 11 Jan)**
 - Activity: Collect project documents, read flow diagrams, confirm scope and pilot services.
 - Milestone: Finalized Problem Statement & Motivation.

- **Week 1 (12 Jan – 18 Jan)**
 - Activity: Literature review and background study (research and summarize).
 - Milestone: Completed Literature Overview & Background Study.
- **Week 2 (19 Jan – 25 Jan)**
 - Activity: Existing system analysis (manual, semi-digital, ERP, queue vendors).
 - Milestone: Completed Existing System Analysis with vendor gaps.
- **Week 3 (26 Jan – 02 Feb)**
 - Activity: Proposed high-level solution, objectives, scope, constraints, KPIs, and feasibility write-up. Final checks.
 - Milestone: Complete Phase-1 document ready for review and submission.

Tentative milestones for Phase-2 & Phase-3

- **Phase 2 (03 Feb – 15 Feb)**
 - Milestones: Architecture diagram, module-wise design, DB/data flow design, and PoC demo.
- **Phase 3 (16 Feb – 03 Mar)**
 - Milestones: Extended prototype / partial implementation, test cases & validation, risk analysis, final report & presentation.

8. Team Structure and Collaboration

Roles & Responsibilities

- **Project Lead / Coordinator**
 - Overall planning, deadlines, progress tracking, liaison with college/staff.
- **Frontend Developer**
 - Build student portal and staff dashboard (React, Zustand, Tailwind, ShadCN).
- **Backend Developer**
 - Implement APIs, workflow engine, queue workers (Node.js, Express, MongoDB, Redis, BullMQ).
- **Chatbot / AI Integrator**
 - Create knowledge base, implement RAG flow and Gemini API integration.
- **QA / Tester**
 - Test flows, record bugs, validate KPIs during pilot.
- **Documenter / Presenter**
 - Prepare Phase-1 report, slides, and demo notes.

Collaboration tools

- **Code & version control:** GitHub (repo + branches + pull requests)
- **Docs & reports:** Google Docs
- **API testing:** Postman

9. Risk and Challenge Analysis

Identified Risks

1. **Technical complexity**
 - Workflow engine, BullMQ scheduling, and RAG chatbot integration can be tricky.
2. **Data / API availability**
 - Gemini API quotas, email provider limits, or unavailable sample data.
3. **Time constraints**
 - Semester timeline is tight for design, PoC and pilot.
4. **Team availability / skills**
 - Members may lack experience in Redis/BullMQ or RAG integration.
5. **Security & privacy**
 - Rate Limiting, Security Attacks (DDOS, SQL etc.)

Mitigation Strategies

- **Technical complexity**
 - Break work into small deliverables; implement a minimal workflow engine (state machine) first.
 - Use existing libraries and examples (BullMQ docs, Mongo patterns).
 - Keep RAG integration modular: start with retrieval + Gemini Q/A calls; expand later.
- **Data / API availability**
 - Use sample or anonymized data during development.
 - Limit Gemini calls during pilot; budget for necessary API usage.
- **Time constraints**
 - Follow the three-phase timeline strictly and deliver a PoC before full pilot.
 - Prioritize core features (queue + workflow + chatbot guidance) and postpone extras.
- **Team availability / skills**
 - Pair less-experienced members with stronger ones for knowledge transfer.
 - Keep tasks small and document patterns; use online tutorials for quick ramp-up.
- **Security & privacy**
 - Use HTTPS, environment variables for keys, and minimal PII storage.
 - Log only required fields and use role-based access.
 - Apply basic encryption for sensitive fields.
- **Pilot participation**
 - Coordinate early with staff, schedule pilot slots, and provide simple user instructions.

- Offer small incentives (helpful demos) and clear contact for issues.
- **ERP integration friction**
 - Keep Phase-1 exports as CSV and simple REST endpoints; avoid DB-level integration.

10. Supervisor Review and Approval

Advisor Feedback:

Supervisor Comments:

Recommendations:

Signature: _____

Date: _____